

LOOP Engine

[1. LOOP Engine – Kompakte System-Referenz für KI-Modelle](#)

[2. Grundsyntax](#)

[3. Referenz der LOOP-Parameter](#)

[4. Condition im LOOP](#)

[5. WHILE](#)

[6. Wann WHILE verwenden?](#)

[7. DO_WHILE](#)

[8. Wann DO_WHILE verwenden?](#)

[9. SQL Polling – vollständiges Beispiel](#)

[10. Ablauf des SQL-Polling-Beispiels](#)

[11. REST Polling](#)

[12. Mehrere aktive Status](#)

[13. Mehrere Endzustände](#)

[14. CONTAINS-Polling](#)

[15. Warum ENGINE → BUFFER → Condition?](#)

[16. maxIterations](#)

[17. timeoutMs](#)

[18. maxIterations und timeoutMs zusammen](#)

[19. WAIT und LOOP-Timeout](#)

[20. Wichtige DO_WHILE-Semantik](#)

[21. Kein BREAK-Step](#)

[22. LOOP Logging](#)

[23. LOOP vs. WAIT](#)

[24. Troubleshooting](#)

[LOOP endet nach einer Iteration](#)

[LOOP erreicht maxIterations](#)

[LOOP erreicht timeoutMs](#)

[Buffer nicht gefunden](#)

[Condition-Wert existiert vor WHILE nicht](#)

[25. Best Practices](#)

[DO_WHILE für Polling](#)

[Technische Antworten in Buffer überführen](#)

[Polling-Intervall sinnvoll wählen](#)

[Immer Safety Guards definieren](#)

[26. Kurzreferenz für KI / RAG](#)

Die LOOP-Engine ermöglicht die wiederholte Ausführung einer Gruppe von AGATE-Schritten.

Die Wiederholung wird über eine `condition` gesteuert und zusätzlich durch `maxIterations` und `timeoutMs` abgesichert.

Typische Anwendungsfälle:

- Polling
- Warten auf asynchrone Backend-Verarbeitung
- wiederholte Statusabfragen

- wiederholte SQL-Abfragen
 - API-Status-Polling
 - technische Synchronisation zwischen Testschritten
-

1. LOOP Engine – Kompakte System-Referenz für KI-Modelle

```
1 AGATE ENGINE: LOOP
2
3 PURPOSE:
4 Repeat nested AGATE steps while a condition remains true.
5
6 TYPE:
7 LOOP
8
9 NO op FIELD.
10
11 REQUIRED:
12 mode
13 condition
14 maxIterations
15 timeoutMs
16 steps
17
18 MODES:
19
20 WHILE
21 - condition before body
22 - 0..N iterations
23
24 DO_WHILE
25 - body before condition
26 - 1..N iterations
27 - preferred when condition value is generated inside the LOOP
28
29 CONDITION:
30
31 LOOP uses the general AGATE Condition Handling syntax.
32
33 LOOP-specific semantics:
34
35 true
36 -> execute another iteration
37
38 false
39 -> terminate LOOP successfully
40
41 SAFETY:
42
43 maxIterations
44 -> maximum number of iterations
45
46 timeoutMs
47 -> maximum total runtime of the complete LOOP
48
49 If a safety guard is reached and condition still requires another
iteration:
50 LOOP = FAILED
51
52 RECOMMENDED POLLING:
53
54 ENGINE EXEC
55 -> ENGINE BUFFER
```

```

56 -> WAIT
57 -> LOOP condition
58
59 NO BREAK STEP.
60
61 DO_WHILE executes the complete child-step list before checking
  condition.
62
63 IMPORTANT:
64 The full Condition syntax is documented separately in:
65 Condition Handling - AGATE DSL

```

2. Grundsyntax

```

1 - type: LOOP
2   mode: DO_WHILE
3   condition: "{B[status]}" != 'READY'"
4   maxIterations: 20
5   timeoutMs: 60000
6
7   steps:
8
9     - type: REST
10      op: EXEC
11      command: demo.status
12      response: status_response
13
14     - type: WAIT
15      value: "2000"

```

3. Referenz der LOOP-Parameter

Parameter	Beschreibung	Erforderlich
type	Muss LOOP sein	Ja
mode	WHILE oder DO_WHILE	Ja
condition	Fortsetzungsbedingung des LOOPS	Ja
maxIterations	Maximale Anzahl Iterationen	Ja
timeoutMs	Maximale Gesamtdauer des LOOPS in Millisekunden	Ja

steps	Child-Steps einer Iteration	Ja
id	Optionaler Step-Identifizier	Nein

4. Condition im LOOP

LOOP verwendet die allgemeine AGATE Condition-Syntax.

Beispiele:

```
1 condition: "{B[count]} == 0"
1 condition: "{B[status]} != 'READY'"
1 condition: "{B[status]} == 'PENDING' OR {B[status]} == 'RUNNING'"
1 condition: "NOT ({B[status]} == 'DONE' OR {B[status]} == 'SUCCESS')"
```

Die vollständige Operator- und Syntax-Referenz befindet sich im separaten Dokument:

Condition Handling – AGATE DSL

Für LOOP ist entscheidend:

```
1 condition true
2 -> nächste Iteration
3
4 condition false
5 -> LOOP erfolgreich beenden
```

5. WHILE

Bei **WHILE** wird die Condition **vor jeder Iteration** geprüft.

```
1 - type: LOOP
2   mode: WHILE
3   condition: "{B[continue_processing]} == 'true'"
4   maxIterations: 10
5   timeoutMs: 30000
6
7   steps:
8
9     - type: CMD
10      op: EXEC
11      command: "echo processing"
```

Ablauf:

```
1 Condition
2   |
3   └─ false
```

```

4      ↓
5      LOOP Ende
6
7      └─ true
8          ↓
9      Child-Steps
10         ↓
11      Condition

```

Ein WHILE-LOOP kann **0 Iterationen** ausführen.

6. Wann WHILE verwenden?

WHILE ist sinnvoll, wenn der Condition-Wert **bereits vor dem LOOP existiert**.

Beispiel:

```

1 - type: BUFFER
2   op: EXEC
3   name: continue_processing
4   value: "true"
5
6 - type: LOOP
7   mode: WHILE
8   condition: "{B[continue_processing]} == 'true'"
9   maxIterations: 10
10  timeoutMs: 30000
11
12  steps:
13    ...

```

Hier existiert:

```

1 B[continue_processing]

```

bereits vor der ersten LOOP-Auswertung.

7. DO_WHILE

Bei **DO_WHILE** wird zuerst der komplette LOOP-Body ausgeführt.

Danach wird die Condition geprüft.

```

1 - type: LOOP
2   mode: DO_WHILE
3   condition: "{B[result_count]} == 0"
4   maxIterations: 30
5   timeoutMs: 130000
6
7   steps:
8
9     - type: SQL
10      op: EXEC
11      command: "SELECT ..."
12      response: result
13
14     - type: SQL
15      op: BUFFER

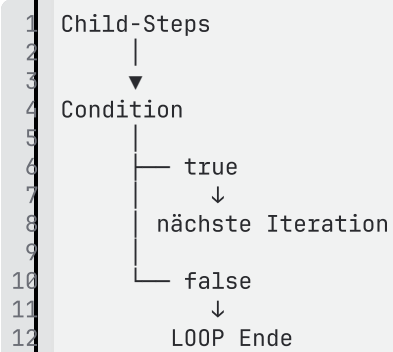
```

```

16     response: result
17     row: "0"
18     column: "0"
19     name: result_count
20
21     - type: WAIT
22       value: "4000"

```

Ablauf:



Ein DO_WHILE führt mindestens **eine Iteration** aus.

8. Wann DO_WHILE verwenden?

DO_WHILE ist der bevorzugte Modus für Polling, wenn der Condition-Wert erst innerhalb des LOOPS erzeugt wird.

Typisches Muster:

```

1 SQL
2 ↓
3 BUFFER
4 ↓
5 Condition

```

Vor der ersten Iteration existiert der Buffer noch nicht.

Daher:

```

1 mode: DO_WHILE

```

9. SQL Polling – vollständiges Beispiel

```

1 - id: wait_until_job_completed
2   type: LOOP
3   mode: DO_WHILE
4   condition: "{B[job_completed_count]} == 0"
5   maxIterations: 30
6   timeoutMs: 130000
7
8   steps:
9
10  - type: SQL
11    op: EXEC
12    command: >

```

```

13     SELECT COUNT(*)
14     FROM DEMO_JOB_STATUS
15     WHERE JOB_ID = '{R[JobId]}'
16           AND STATUS = 'COMPLETED'
17     response: job_status_response
18
19 - type: SQL
20   op: BUFFER
21   response: job_status_response
22   row: "0"
23   column: "0"
24   name: job_completed_count
25
26 - type: WAIT
27   value: "4000"

```

10. Ablauf des SQL-Polling-Beispiels

Iteration 1:

```

1 | SQL -> count 0
2 | BUFFER -> job_completed_count = 0
3 | WAIT 4000
4 | Condition: 0 == 0 -> true

```

Weitere Iteration.

Iteration 2:

```

1 | SQL -> count 0
2 | BUFFER -> 0
3 | WAIT
4 | Condition -> true

```

Weitere Iteration.

Spätere Iteration:

```

1 | SQL -> count 1
2 | BUFFER -> 1
3 | WAIT
4 | Condition: 1 == 0 -> false

```

LOOP Ende.

11. REST Polling

```

1 - type: LOOP
2   mode: DO_WHILE
3   condition: "{B[api_status]} != 'READY'"
4   maxIterations: 20
5   timeoutMs: 60000
6
7   steps:
8
9     - type: REST
10       op: EXEC
11       command: demo.status
12       response: status_response

```

```
13
14     # Status über die passende BUFFER-Funktion
15     # nach B[api_status] übernehmen
16
17     - type: WAIT
18       value: "2000"
```

Möglicher Verlauf:

```
1 PENDING
2 -> weiter
3
4 PROCESSING
5 -> weiter
6
7 READY
8 -> LOOP Ende
```

12. Mehrere aktive Status

```
1 - type: LOOP
2   mode: DO_WHILE
3   condition: "{B[status]} == 'PENDING' OR {B[status]} == 'RUNNING'"
4   maxIterations: 20
5   timeoutMs: 60000
6
7   steps:
8     ...
```

Der LOOP läuft bei:

```
1 PENDING
2 RUNNING
```

weiter.

Bei beispielsweise:

```
1 DONE
2 READY
3 FAILED
```

ist die Condition false und der LOOP endet.

13. Mehrere Endzustände

```
1 - type: LOOP
2   mode: DO_WHILE
3   condition: "NOT ({B[status]} == 'DONE' OR {B[status]} == 'SUCCESS')"
4   maxIterations: 20
5   timeoutMs: 60000
6
7   steps:
8     ...
```

Der LOOP endet bei:

```
1 DONE
```


14. CONTAINS-Polling

```

1 - type: LOOP
2   mode: DO_WHILE
3   condition: "NOT {B[log_output]} CONTAINS 'PROCESS FINISHED'"
4   maxIterations: 20
5   timeoutMs: 60000
6
7   steps:
8
9     - type: CMD
10      op: EXEC
11      command: "demo-status-command"
12
13     - type: WAIT
14      value: "2000"

```

Der LOOP endet, sobald der erwartete Text vorhanden ist.

15. Warum ENGINE → BUFFER → Condition?

Empfohlen:

```

1 ENGINE
2   |
3   ▼
4 RESPONSE
5   |
6   ▼
7 BUFFER
8   |
9   ▼
10 CONDITION

```

Beispiel:

```

1 - type: SQL
2   op: EXEC
3   response: result
4
5 - type: SQL
6   op: BUFFER
7   response: result
8   row: "0"
9   column: "0"
10  name: result_count

```

Dann:

```

1 condition: "{B[result_count]} == 0"

```

Damit bleibt LOOP unabhängig von der technischen Quelle des Werts.

Der Wert kann aus:

```

1 SQL

```

```
2 REST
3 SOAP
4 CMD
5 OC
6 FILE
7 ...
```

stammen.

16. maxIterations

`maxIterations` schützt vor Endlosschleifen.

```
1 maxIterations: 30
```

Der LOOP darf maximal 30 Iterationen ausführen.

Wenn die Condition danach immer noch eine weitere Iteration verlangt, schlägt der LOOP fehl.

`maxIterations` ist ein Safety Guard und kein regulärer Exit.

17. timeoutMs

`timeoutMs` begrenzt die maximale Gesamtdauer des LOOPS.

```
1 timeoutMs: 130000
```

Die Zeit umfasst:

```
1 alle Iterationen
2 +
3 SQL
4 +
5 REST
6 +
7 SOAP
8 +
9 CMD
10 +
11 WAIT
12 +
13 CALL
14 +
15 weitere Child-Steps
```

18. maxIterations und timeoutMs zusammen

Beide Parameter schützen gegen unterschiedliche Fehler.

`maxIterations`:

```
1 Schutz gegen sehr schnelle Endlosschleifen
```

`timeoutMs`:

```
1 | Schutz gegen sehr lange LOOP-Ausführungen
```

Beide sollen bei jedem LOOP definiert sein.

19. WAIT und LOOP-Timeout

Beispiel:

```
1 | maxIterations: 30
2 | timeoutMs: 130000
```

mit:

```
1 | - type: WAIT
2 |   value: "4000"
```

Maximale reine WAIT-Zeit:

```
1 | 30 × 4000 ms
2 | =
3 | 120000 ms
```

Zusätzlich benötigen:

```
1 | SQL
2 | BUFFER
3 | Condition
4 | Logging
```

Zeit.

Daher ist:

```
1 | timeoutMs: 130000
```

sinnvoller als exakt:

```
1 | timeoutMs: 120000
```

20. Wichtige DO_WHILE-Semantik

Bei DO_WHILE wird der **gesamte Child-Step-Block** ausgeführt, bevor die Condition geprüft wird.

Beispiel:

```
1 | steps:
2 |
3 |   - type: SQL
4 |     ...
5 |
6 |   - type: SQL
7 |     op: BUFFER
8 |     ...
9 |
```

```
10 - type: WAIT
11   value: "4000"
```

Wenn der SQL-Step bereits den Zielzustand findet:

```
1 SQL -> 1
2 BUFFER -> 1
```

wird trotzdem:

```
1 WAIT 4000
```

ausgeführt.

Erst danach:

```
1 Condition -> false
2 LOOP Ende
```

21. Kein BREAK-Step

Die aktuelle LOOP-Version besitzt keinen separaten:

```
1 - type: BREAK
```

Step.

Die Steuerung erfolgt ausschließlich über:

```
1 mode
2 +
3 condition
```

Beispiel einer BREAK-Denkweise:

```
1 BREAK wenn count > 0
```

wird als Fortsetzungsbedingung modelliert:

```
1 condition: "{B[count]} == 0"
```

Solange `count == 0` gilt, läuft der LOOP.

Sobald `count != 0` gilt, endet er.

22. LOOP Logging

Typischer Start:

```
1 >>> LOOP      : mode=D0_WHILE | maxIterations=30 | timeoutMs=120000
2 >>> LOOP ITER : 1/30
```

Nach einer Iteration:

```
1 >>> LOOP COND : {B[result_count]} == 0 -> true
2 >>> LOOP ITER : 2/30
```

Beim regulären Ende:

```
1 >>> LOOP COND : {B[result_count]} == 0 -> false
2 >>> LOOP END : iterations=4 | duration=16082 ms
```

23. LOOP vs. WAIT

WAIT	LOOP
wartet feste Dauer	wartet auf einen Zustand
keine Wiederholung	wiederholt Child-Steps
keine Condition	Condition-gesteuert
kann unnötig lange warten	kann frühzeitig enden
kann zu kurz sein	prüft tatsächlichen Zustand

Statisch:

```
1 - type: WAIT
2   value: "15000"
```

bedeutet:

```
1 Immer 15 Sekunden warten.
```

LOOP:

```
1 - type: LOOP
2   mode: DO_WHILE
3   condition: "{B[ready]} != 'true'"
```

bedeutet:

```
1 Wiederholt prüfen und erst weitergehen,
2 wenn ready == true.
```

24. Troubleshooting

LOOP endet nach einer Iteration

Prüfen:

```
1 >>> LOOP COND ... -> false
```

Wenn die Condition `false` ergibt, endet der LOOP korrekt.

LOOP erreicht maxIterations

Prüfen:

- Wird der Buffer aktualisiert?
 - Ist der richtige Buffer referenziert?
 - Wird der erwartete Zustand überhaupt erreicht?
 - Ist die Condition korrekt formuliert?
-

LOOP erreicht timeoutMs

Prüfen:

- Polling-Intervall
 - Child-Step-Laufzeiten
 - Anzahl Iterationen
 - Backend-Verarbeitungszeit
 - Engine-spezifische Timeouts
-

Buffer nicht gefunden

Bei beispielsweise:

```
1 | SQL response was not found
```

prüfen, ob EXEC und BUFFER denselben Response-Namen verwenden.

```
1 | - type: SQL
2 |   op: EXEC
3 |   response: demo_response
4 |
5 | - type: SQL
6 |   op: BUFFER
7 |   response: demo_response
```

Condition-Wert existiert vor WHILE nicht

Wenn der Buffer erst im LOOP erzeugt wird:

Nicht:

```
1 | mode: WHILE
```

sondern:

```
1 mode: DO_WHILE
```

verwenden.

25. Best Practices

DO_WHILE für Polling

Wenn der Status im LOOP gelesen wird:

```
1 mode: DO_WHILE
```

verwenden.

Technische Antworten in Buffer überführen

Bevorzugt:

```
1 ENGINE
2 -> BUFFER
3 -> CONDITION
```

Polling-Intervall sinnvoll wählen

Beispielsweise:

```
1 value: "1000"
```

```
1 value: "2000"
```

```
1 value: "4000"
```

abhängig vom Zielsystem.

Immer Safety Guards definieren

```
1 maxIterations: ...
2 timeoutMs: ...
```

26. Kurzreferenz für KI / RAG

```
1 AGATE LOOP ENGINE
2
3 TYPE:
4 LOOP
5
6 PURPOSE:
7 Repeated execution of nested AGATE steps.
8
9 REQUIRED:
10 mode
11 condition
12 maxIterations
13 timeoutMs
14 steps
15
```

```
16 MODE WHILE:
17 condition first
18 0..N executions
19
20 MODE DO_WHILE:
21 body first
22 1..N executions
23
24 RECOMMENDED FOR POLLING:
25 DO_WHILE
26
27 LOOP CONDITION SEMANTICS:
28
29 true
30 -> another iteration
31
32 false
33 -> successful LOOP exit
34
35 CONDITION LANGUAGE:
36 Use global AGATE Condition Handling.
37 Do not redefine Condition syntax inside LOOP.
38
39 PREFERRED PATTERN:
40
41 ENGINE EXEC
42 -> ENGINE BUFFER
43 -> WAIT
44 -> LOOP CONDITION
45
46 SAFETY:
47
48 maxIterations
49 timeoutMs
50
51 NO BREAK STEP.
52
53 DO_WHILE:
54 complete body executes before condition.
55
56 Therefore:
57 ENGINE
58 BUFFER
59 WAIT
60 CONDITION
61
62 means WAIT also executes in the final iteration.
63
64 AI GENERATION:
65
66 1. Prefer DO_WHILE for polling.
67 2. Generate maxIterations.
68 3. Generate timeoutMs.
69 4. Prefer ENGINE -> BUFFER -> CONDITION.
70 5. Keep LOOP conditions simple.
71 6. Refer to Condition Handling documentation for supported operators.
```